WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

类型推断技术及仓颉语言实践

主讲人：史磊



目录

- 类型推断技术简介
- 仓颉的类型推断实践
- 未来可能的改进点

类型安全（type safety）

- 简单来说：
  - 一个类型良好的程序从不“出错”
- 更严谨地说：
  - 进展性：一个类型良好的程序不应该“卡住”，每个表达式要么已经是一个值，要么可以朝一个值进行演算，不会出现无法处理的未定义状态。
  - 保持性：每步运算后每个表达式的类型都不变。

类型安全（type safety）

- 实际应用中，可以简单理解为
  类型安全 ≈ 内存安全

格式不匹配!

```
let v: Float32 = Int64(123)
```

操作不匹配!

```
func f(s: String) {
    s.replace(...)
}

let s = f(1)
```

类型推断是在做什么？

- 确定每个声明和表达式的类型
  - 可以省略的变量类型、函数返回类型、泛型实参类型等等
  - 分支、循环、字面量等并不涉及类型标注的各种表达式
  - 保证类型安全

```
1   //  v: String
2   let v = "Hello World"
3
4   //          Some<Int64>(1)
5   let maybe = Some(1)
6
7   //   pos(x:Int): bool
8   func pos(x:Int) {
9       return x > 0
10  }
11
```

类型推断是在做什么？

- 确定每个名字对应的声明
  - 主要指函数有重载的情形
  - 帮助后续的优化、链接等等

```
1   func foo(x:Int) {}    // foo1
2   func foo(x:String) {} // foo2
3
4   let v = foo(123)      // foo1
```

Hindley-Milner 风格的类型推断

- 大致思路
  - 给每个未知类型创建一个变元
  - 建立变元间的相等关系
  - 解“方程组”

- 全局推断
  - 程序中所有信息都可以用来求解
  - 没有“顺序依赖”
  - 甚至函数的参数类型/泛型参数也可以推断

Rust

```rust
fn f(x:u16) {}

fn main() {
    let v = 1;
    f(v);
}
```

v: Tv
Tv = u16

子类关系和命名子类关系带来的困难

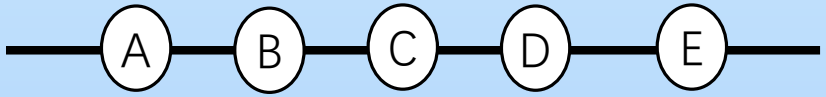
- 右边等价的程序，在大多数不允许隐式转换的工业语言中都无法通过类型检查
  - 在第4行时v的类型就必须确定了
  - 无法通过第5行的信息“回头”求解
- Hindley-Milner 风格的类型推断这么nice，为什么大家不都用它？

Rust

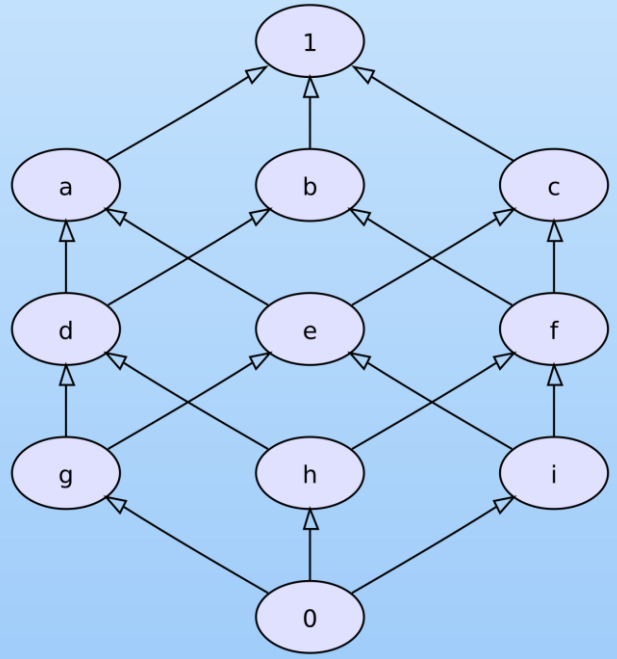
```rust
1  fn f(x:u16) {}
2
3  fn main() {
4      let v = 1;
5      f(v);
6  }
```

子类关系和命名子类关系带来的困难

- 没有子类关系的类型系统



- 有子类关系的类型系统



仓颉 Cangjie

子类关系和命名子类关系带来的困难

- 没有子类关系的“方程组”：

Ta = Tb
Tc = Td
Te = String
Int->Tf = Tg->Int

- 有子类关系的“不等式组”：

Ta <: Tb
Tc <: Td
Te <: String
Int->Tf <: Tg->Int

仓颉 Cangjie

子类关系和命名子类关系带来的困难

- 结构化子类关系下求解:

```
Ta <: {f1: Int}
Ta <: {f2: String}
=>
Ta = {f1: Int} /\ {f2: String}
   = {f1: Int, f2: String}
```

- 命名子类关系下求解(?):

```
class A {let f1: Int}
class B {let f2: String}

Ta <: A
Ta <: B
=>
Ta = ???
```

子类关系和命名子类关系带来的困难

- 子类关系 + 命名子类关系
  => 数学性质过于糟糕
  => 学术界没有兴趣

局部类型推断

- 直到

  Pierce B C, Turner D N. **Local type inference**[J]. ACM Transactions on Programming Languages and Systems (TOPLAS), 2000, 22(1): 1-44.

局部类型推断

- 大致思路
  - 只推断泛型实参类型，不推断函数的参数类型
  - 只从相邻语法节点获取类型信息，泛型实参只局部求解，不全局求解
  - 类型信息自顶向下和自底向上双向传播

- 严格使用这个思路的语言
  - 仓颉
  - Swift
  - 其他工业语言大多没有完全做到“双向传播”

局部类型推断

- 大致思路
  - 只推断泛型实参类型，不推断函数的参数类型
  - 只从相邻语法节点获取类型信息，泛型实参只局部求解，不全局求解
  - 类型信息自顶向下和自底向上双向传播

Accept

```
1   func id<T>(x:T) {x}
2
3   //      id<String>
4   let v = id("1")
```

Reject

```
1   // tryPrint<T>(x: Option<T>): Unit where T <: ToString
2   func tryPrint(x) {
3       if (let Some(v) = x) {
4           println("Has value ${v}")
5       }
6   }
```

局部类型推断

- 大致思路
  - 只推断泛型实参类型，不推断函数的参数类型
  - 只从相邻语法节点获取类型信息，泛型实参只局部求解，不全局求解
  - 类型信息自顶向下和自底向上双向传播

Accept

```
1   func id<T>(x:T) {x}
2
3   //            id<Int32>
4   let v:Int32 = id(1)
```

Reject

```
1   func id<T>(x:T) {x}
2
3   //      id<Int32>
4   let v = id(1)
5   let v2: Int32 = v
```

局部类型推断

- 大致思路
  - 只推断泛型实参类型，不推断函数的参数类型
  - 只从相邻语法节点获取类型信息，泛型实参只局部求解，不全局求解
  - 类型信息自顶向下和自底向上双向传播

Accept

```
1  func id<T>(x: T): T {
2      x
3  }
4
5  let s = id("hello")     // will infer id<String>
```

Accept

```
1  func handleByte(b: Byte) {
2  }
3
4  func f() {
5      let v: Int32 = 1 // 1 inferred to Int32
6      handleByte(2)    // 2 inferred to Byte
7  }
```

局部类型推断

- 大致思路
  - 只推断泛型实参类型，不推断函数的参数类型
  - 只从相邻语法节点获取类型信息，泛型实参只局部求解，不全局求解
  - 类型信息自顶向下和自底向上双向传播

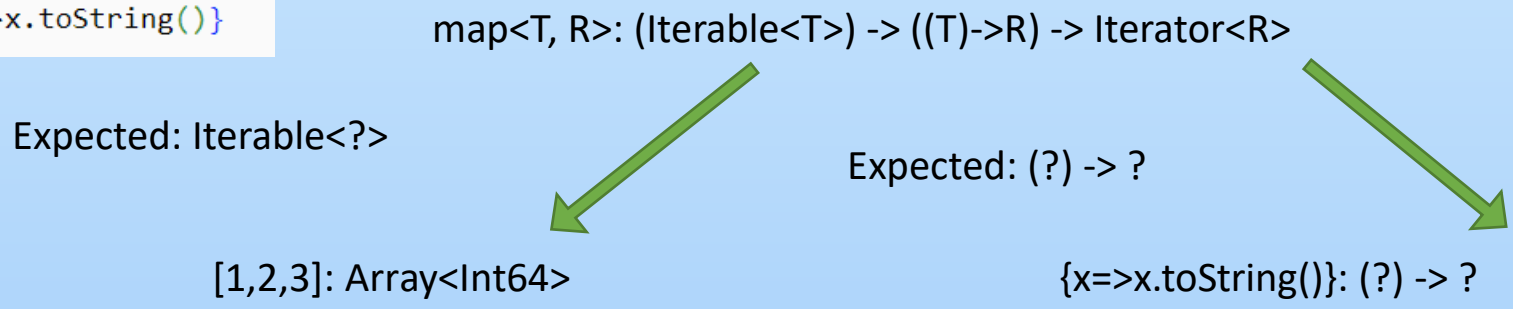
Accept

```
1  func narrow(x: Int64):Byte {
2      let v: Byte = if (x >= 0) {
3          1          // 1 inferred to Byte
4      } else {
5          -1         // -1 inferred to Byte
6      }
7      return v
8  }
```

一些难点 - 难以求解的泛型参数

- 一次自顶向下/自底向上无法解决

```
1    let it = [1, 2, 3] |> map{x=>x.toString()}
```

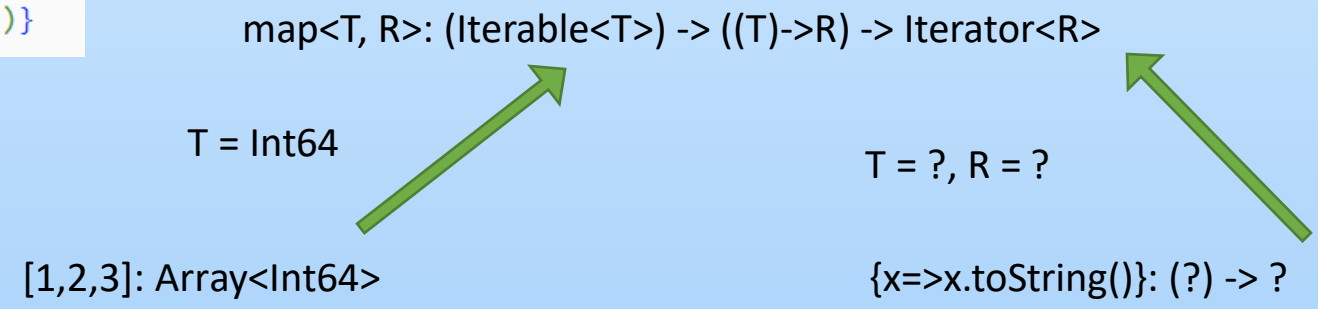


仓颉 Cangjie

一些难点 - 难以求解的泛型参数

- 一次自顶向下/自底向上无法解决

```
1    let it = [1, 2, 3] |> map{x=>x.toString()}
```

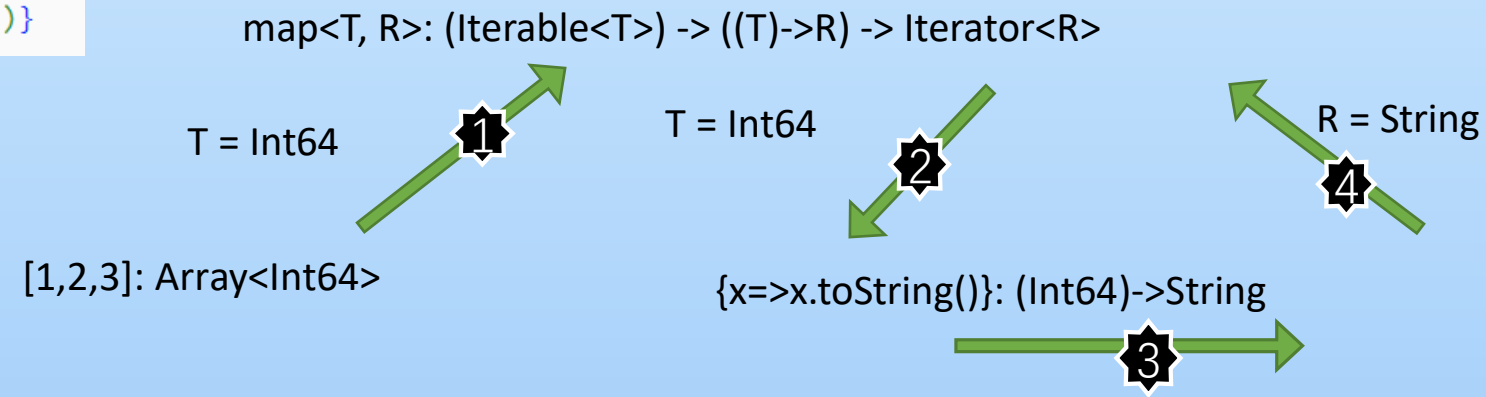


仓颉 Cangjie

一些难点 - 难以求解的泛型参数

- 一次自顶向下/自底向上无法解决
  - 反复迭代直到解出所有泛型参数/没有新信息

```
1    let it = [1, 2, 3] |> map{x=>x.toString()}
```



仓颉 Cangjie

一些难点 - 嵌套的重载调用

- 双向类型推断
  - 期望类型影响重载决议结果
  - 外层决议结果影响内层决议结果
  - 内层参数类型同样影响决议结果
  - 是否要搜索指数多种组合?

```
1   func widen(x: Int8):Int16 {
2       Int16(x)
3   }
4
5   func widen(x: Int16):Int32 {
6       Int32(x)
7   }
8
9   main() {
10      let v: Int32 = widen(widen(1))      // 1 inferred to Int8
11  }
```

一些难点 - 嵌套的重载调用

- 为什么要考虑这种奇怪的程序?
  - 复杂的算术表达式
  - 比较大的UI组件

```
from ohos import base.*
from ohos import component.*
from ohos import state_manage.*
from ohos import state_macro_manage.*
from std import collection.*

@Entry
@Component
class button_case {
    func render() {
        Column(30) {
            Text("button_case_004: create: 入参嵌套Button组件")
            Button() {
                Column() {
                    Text("第一层Button")
                    Button() {
                        Column() {
                            Text("第二层Button")
                            Button() {
                                Column() {
                                    Text("第三层Button")
                                    Button() {
                                        Column() {
                                            Text("第四层Button")
                                            Button() {
                                                Column() {
                                                    Text("第五层Button")
                                                    Button() {
                                                        Column() {
                                                            Text("第六层Button")
```

```
Swift
1    let a: Double = -(1 + 2) + -(3 + 4) + -(5)
```

Swift 编译这个会超时

宏展开以后:

```
class column_test <: CustomView {
    public init(a: String) {
        println(a)
    }

    // this.observeComponentCreation({elmtId, isInitialRender ⇒ Column("111") })
    public func render() {
        this.observeComponentCreation({elmtId, isInitialRender ⇒
            Column() { this.observeComponentCreation({elmtId, isInitialRender ⇒
                Column() { this.observeComponentCreation( {elmtId, isInitialRender ⇒
                    Column() { this.observeComponentCreation({elmtId, isInitialRender ⇒
                        Column() { this.observeComponentCreation({elmtId, isInitialRender ⇒
                            Column() {this.observeComponentCreation({elmtId, isInitialRender ⇒
                                Column() {this.observeComponentCreation({elmtId, isInitialRender ⇒
                                    Column() {this.observeComponentCreation({elmtId, isInitialRender =
                                        Column() {this.observeComponentCreation({elmtId, isInitialRend
                                            Column() {this.observeComponentCreation({elmtId, isInitial
                                                Column() {this.observeComponentCreation({elmtId, isIni
                                                    Column() { this.observeComponentCreation({elmtId,
                                                        Column() {this.observeComponentCreation({elmtI
                                                            Column("111")
```

一些难点 - 嵌套的重载调用

- 重载决议取决于哪些信息？
  - 当前的表达式（不变）
  - 对当前表达式的期望类型（组合有限）
  - 当前上下文中的符号表（几乎不变）
- 给定期望类型，有最优子结构 => 动态规划！

未来可能的改进点 - 一些全局类型推断能力

没有上下文信息也不用标lambda参数类型

```
let f = { x => x + 1 }
```

任何情况都不用标注None的类型

```
func pos(i: Int) {
    var maybe = None
    if (i > 0) {
        maybe = Some(i)
    }
}

```

推断递归函数的返回类型

```
enum Expr {
    | Num(Int64)
    | Add(Expr, Expr)
}

func eval(e: Expr) {
    match(e) {
        case Num(v) => return v
        case Add(e1, e2) => return eval(e1) + eval(e2)
    }
}

let v = eval(Add(Num(1), Num(1)))
```

未来可能的改进点 - Flow Typing

根据上下文自动向下合法转型

```
1   open class Animal {}
2
3   class Dog <: Animal {
4       func woof() { println("woof") }
5   }
6
7   func foo(a: Animal) {
8       if (a is Dog) {
9           a.woof()
10      }
11  }
```

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

